CSN15101 · SOFTWARE ENGINEERING AND PROJECT MANAGEMENT

E-Commerce & Order Management System

Mid-Semester Project Report

Member	Registration no.	Role
Tarush Banke	20243293	Project lead · Database architect
Tanishk Varshney	20243288	Back-end & database logic
Tanishq Mahajan	20243289	Front end & ML · QA lead

Group: TarushTanishkTanishq

Instructor / Supervisor: Komal Gupta

Submission: 16 October 2026 · Release v1.0 (mid-semester)

Repository: github.com/TarushB/SEPM-Ecommerce-Order-Management

Contents

1 Problem statement and motivation · 2 Objectives · 3 Scope · 4 Stakeholders · 5 Requirements specification · 6 Requirements model · 7 System design · 8 Technology stack · 9 Implementation progress · 10 Testing performed · 11 Project-management status · 12 Individual contributions · 13 Challenges and solutions · 14 Work planned for the remaining semester · 15 AI-tool disclosure · 16 References

1. Problem statement and motivation

Online marketplaces handle thousands of orders a day across many sellers, customers, payment methods and delivery routes. The public Olist dataset (≈100k Brazilian orders, 2016–2018) arrives as nine flat CSV files. Kept that way, the data has **redundancy and update anomalies**: one city name is repeated 17,802 times, and renaming one category touches 11,116 rows. It has **no access control** and **no way to spot orders about to go wrong**. Late delivery is the strongest driver of bad reviews: late orders average 2.57★ against 4.29★ for on-time ones.

We build a database-centric order management system that stores this data correctly, lets different staff roles work with it safely through a web interface that **shows every SQL statement it runs**, and uses the data to **predict late deliveries** at checkout.

2. Project objectives

#	Objective	Measured by	Status
O1	Model the domain and store it in a normalized relational database	ER model, keys, FDs, 16 tables in BCNF	Met
O2	Enforce business rules and security inside the DBMS	Constraints, 6 triggers, 5 procedures, 5 roles + RLS	Met
O3	Web UI for insert, search, update, delete and reports, showing the SQL executed	CRUD for 6 entities, 16 reports, SQL panel on every page	Met
O4	Demonstrate transactions, ACID and concurrency control	Transactions lab + SQL demo scripts	Met
O5	One useful AI/ML feature integrated with the DB and UI	Late-delivery classifier (test ROC-AUC 0.72) + delivery-time regressor	Met
O6	Run the work as a managed software project	Sprints, task board, risk register, minutes, Git history, tests	Met

3. Scope of the project

Features included: order, product, customer, seller, review and category management (CRUD); faceted order search; role-based access for five staff roles; 16 SQL reports with EXPLAIN and CSV export; read-only SQL console; transactions and concurrency lab; late-delivery risk and delivery-time prediction; ER model and live schema viewer.

Features excluded (this release): real payment processing, a customer-facing storefront, e-mail notifications, cloud deployment, mobile app.

Assumptions: the nine Olist CSVs are available locally; PostgreSQL 16 with `pgcrypto` is installed; users are internal staff.

Constraints: three-member team; about two weeks for the mid-semester release; free/open-source tools only; must run offline on a Windows laptop; the dataset is not committed to Git (licence and size).

4. Stakeholder identification

Stakeholder	Interest	Influence	How the system serves them
Operations manager	Place and track orders, act on delivery risk	High	CRUD, stored procedures, ML page
Seller	Manage own orders, keep data private	Medium	Row-level security; status updates
Data analyst	Reliable reports	Medium	16 reports, EXPLAIN, CSV; no exact locations
Customer support	Look up orders, answer reviews	Medium	Privacy view, review answering
Administrator	Users and access	High	User management, least-privilege roles
End customers (indirect)	On-time delivery, privacy	Low	Better delivery estimates; data minimisation
Course instructor / evaluator	Correct, demonstrable DBMS + SE practice	High	SQL panel, ER page, test evidence, documents
Development team	Deliver on time, share work fairly	High	Layer ownership, Scrum, Git

5. Requirements specification

Full SRS (IEEE 830 structure): `02_Requirements/SRS.pdf`. Summary with **MoSCoW priority**:

ID	Functional requirement	Priority
FR-1/2	Login with bcrypt check in the DB; each request runs under the user's PostgreSQL role	Must
FR-3/4	Create, search, update, delete orders, products, customers, sellers, reviews, categories	Must
FR-5	Place an order atomically (order + items + payment) via `sp_place_order`	Must
FR-6/7	Order status only moves forward; every change audited	Must
FR-8/9/10	No delete of delivered/paid orders; reviews only after delivery; stock never negative	Should
FR-11	Faceted order search with per-option row counts	Should
FR-12/13	16 reports with EXPLAIN and CSV; SQL shown for every action	Must
FR-14	ER model and live schema in the UI	Must
FR-15	Demonstrate atomicity, consistency, isolation, lost update	Must
FR-16	Sellers see only their own orders; analysts cannot see coordinates	Must
FR-17/18	Predict late-delivery risk and delivery time; "predict only" rolls back	Should
FR-19/20	Admin creates users; read-only SQL console	Could

ID	Non-functional requirement	Target
NFR-1/2	Security: least privilege, bound parameters, passwords stay in the DB	No privilege escalation or injection (TC-16–21)
NFR-3/4	Integrity and reliability: BCNF + constraints; one transaction per request	All rules rejected inside the DB
NFR-5	Performance	Order search < 1 s on 99k orders

ID	Non-functional requirement	Target
NFR-6/7/8	Usability, maintainability, portability	Three scripts to build and run; one SQL file per concern; Win/macOS/Linux
NFR-9	Transparency	SQL shown = SQL executed
NFR-10	ML validity	Time-ordered validation; report ROC-AUC, PR-AUC, recall

6. Requirements model

The use-case diagram and full use-case descriptions are in `02_Requirements/Use_Cases_or_User_Stories.pdf`.

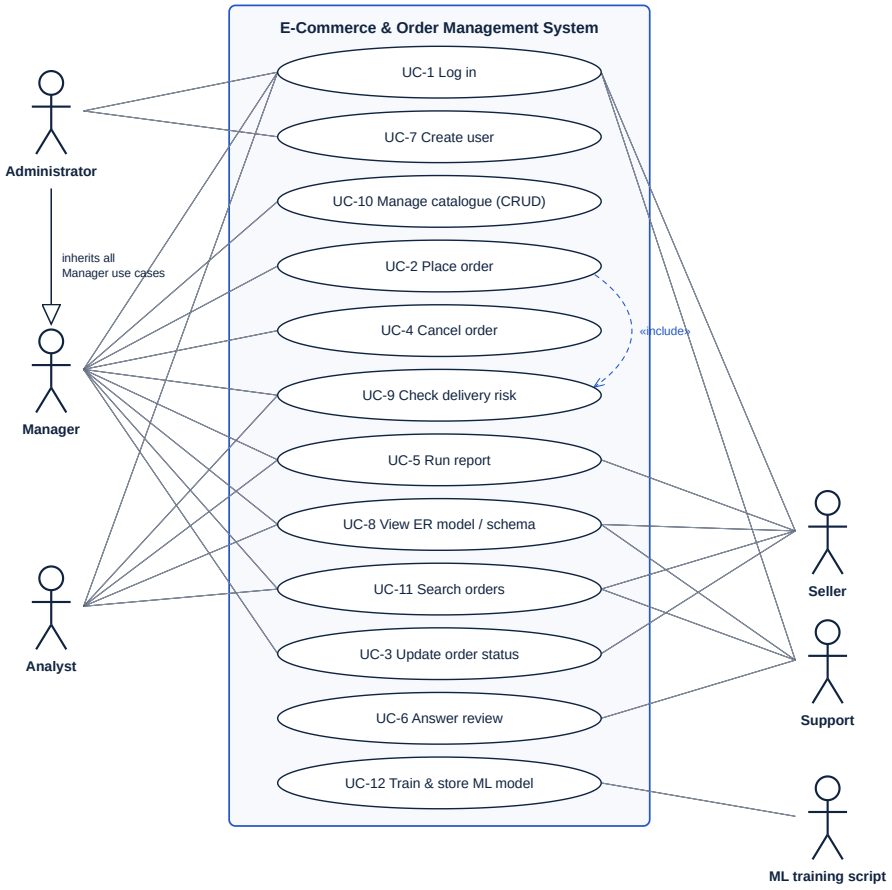


Figure 1: Use-case diagram (12 use cases, 6 actors).

Sample user stories with acceptance criteria

ID	Story	Acceptance criteria
US-1	As a manager, I want to place a multi-item order in one step so that it is never half-saved.	Order, items and payment saved together; any failure saves nothing (TC-11–13)
US-2	As a seller, I want to see only my orders.	0 foreign orders visible, enforced by PostgreSQL RLS (TC-17)
US-3	As a manager, I want status changes validated and logged.	Backward moves rejected; audit row per change (TC-06–08)
US-6	As a manager, I want a late-delivery risk at checkout.	Risk and predicted days shown and stored (TC-30)

7. System design

7.1 Architecture

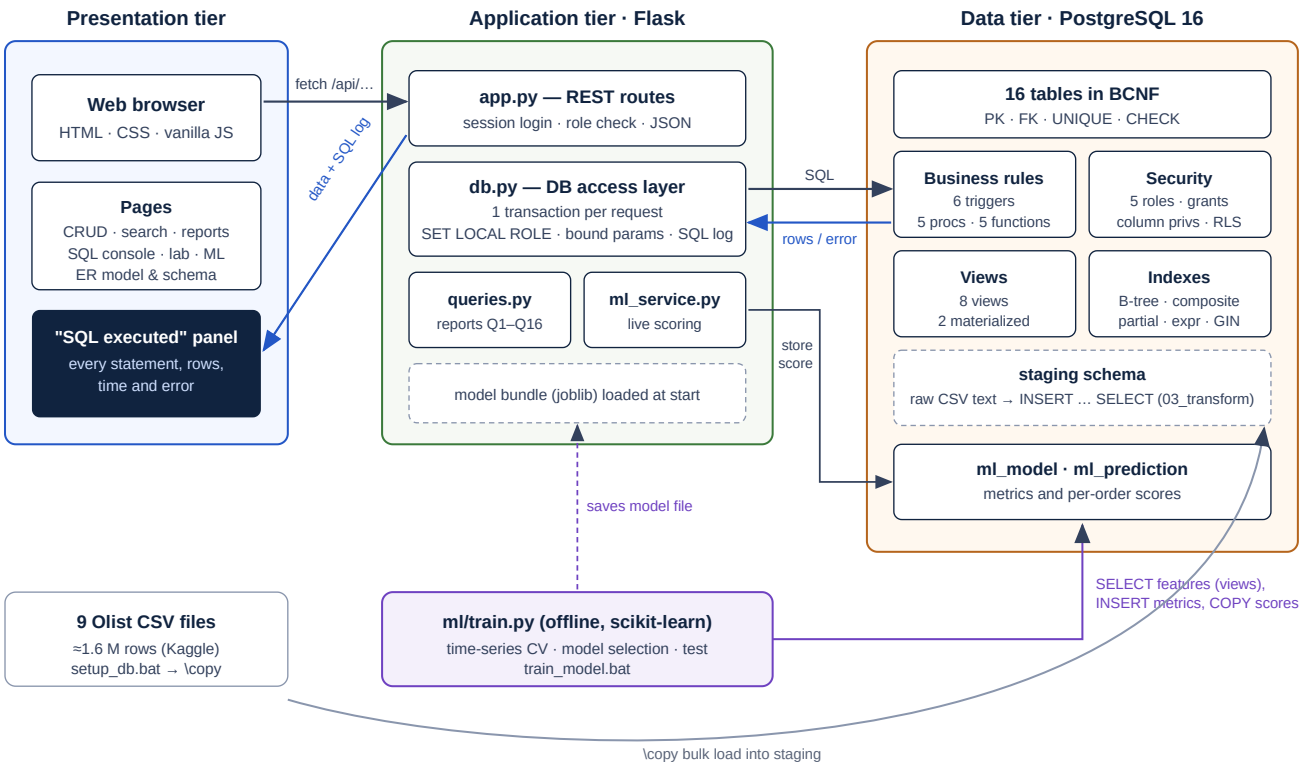


Figure 2: Three-tier architecture. Business rules and access control live in the database tier.

Every request is one transaction: `BEGIN → SET LOCAL ROLE olist_<role> → parameterised SQL → COMMIT/ROLLBACK`. The app logs in as `olist_app`, which has no data privileges of its own, so **PostgreSQL, not the app, enforces permissions**. psycopg's client-side binding means the SQL listed in the UI is exactly what the server ran.

7.2 Module description

Module	Responsibility
`db/01 – 05`	Schema with PK/FK/UNIQUE/CHECK; CSV staging load and cleaning; indexes; views and materialized views
`db/06 – 08`	Triggers, procedures, functions; roles, grants, column privileges, RLS, demo users
`db/09 – 15`	Report queries, ACID/concurrency/DDL/index/normalization demos, automated test suite
`backend/db.py`, `app.py`, `queries.py`	Per-request transaction and SQL log; REST API; report catalogue
`backend/ml_service.py`, `ml/`	Live scoring; offline training with time-series CV
`frontend/`	Single-page UI with the SQL panel and ER/schema page

7.3 Database design

16 relations in BCNF: `state`, `zip_code`, `customer`, `customer_account`, `seller`, `category`, `product`, `orders`, `order_item` (weak), `payment_type`, `payment` (weak), `review`, `order_status_log`, `ml_model`, `ml_prediction`, `app_user`. Normalization went 1NF (items and payments split out), 2NF (order attributes off the item key), 3NF (`zip → city/state`, `category → English name`, `payment_type → description`), then BCNF (`category` has two candidate keys, both determinants). `review` needs the composite key `(review_id, order_id)` because `review_id` alone repeats 827 times in the data. ER diagram, schema, FDs, views, indexes and security: `03_Design/Database_Design.pdf`.

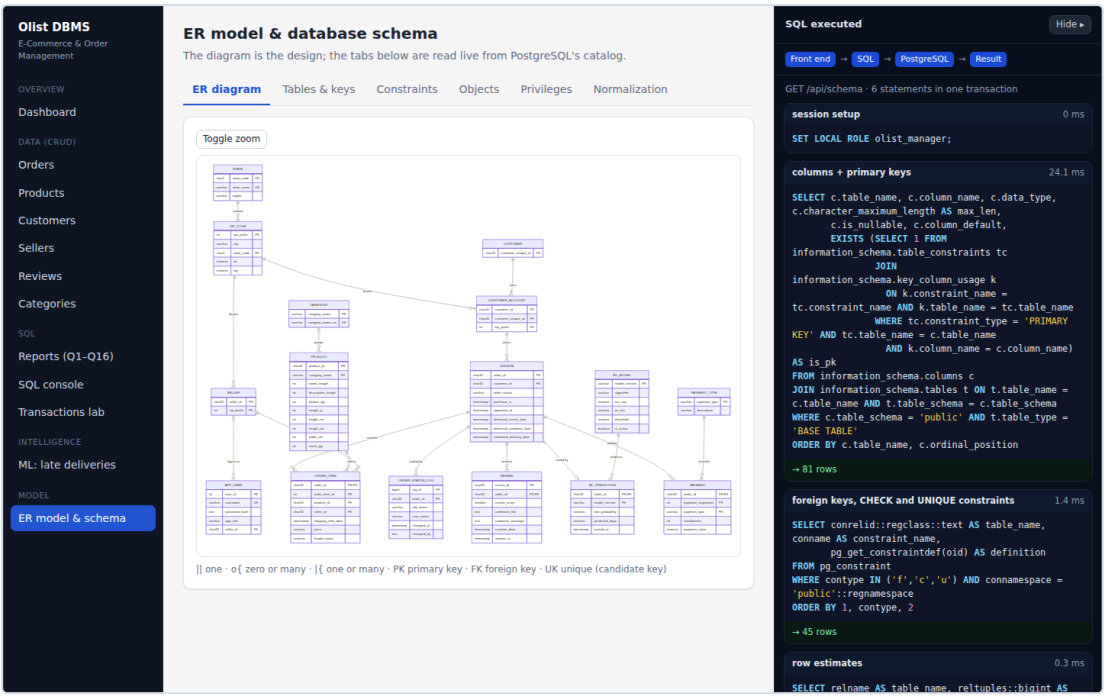
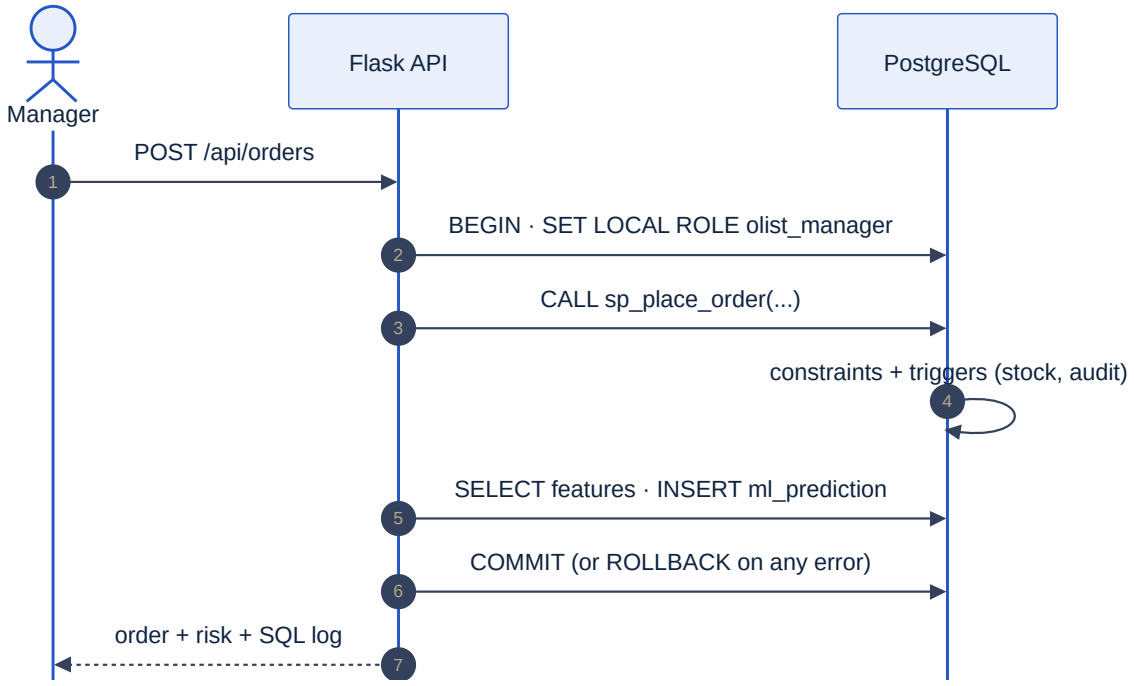


Figure 3: ER model as shown inside the application ("ER model & schema" page).

7.4 Important UML diagrams

`03_Design/UML_Diagrams.pdf` contains the order-status **state machine**, **sequence diagrams** for login/RBAC and for a seller status update, the **activity diagram** of the ML pipeline, and a **class diagram** of the application modules.



8. Technology stack and justification

Layer	Technology	Why
Database	PostgreSQL 16, PL/pgSQL, pgcrypto	Full SQL (window functions, CTEs, RLS, GIN full text, materialized views); free; course focus
Back end	Python 3, Flask 3, psycopg 3	Small, explicit; raw SQL so it can be shown; client-side binding
Front end	HTML, CSS, vanilla JS; Mermaid.js	No build step for evaluators; Mermaid renders the ER diagram
ML	pandas, scikit-learn, joblib (LightGBM optional)	Standard, well-documented; time-series CV support
Tooling	Git + GitHub, VS Code, psql, `.bat` scripts	Version control and evidence; one-click setup on Windows

9. Implementation progress

9.1 Features completed

All objectives O1–O6 are implemented and pass their tests: database with ≈560k loaded rows, 6 triggers, 5 procedures, 5 functions, 8 views + 2 materialized views, 5 roles with RLS and column privileges; Flask API with CRUD for 6 entities, faceted search, 16 reports, SQL console, transactions lab and schema endpoint; ML late-delivery classifier and delivery-time regressor integrated with the database and the order form.

ML results (held-out test months Jun–Aug 2018, late rate 5.5%): ROC-AUC **0.721**, PR-AUC **0.112** (2× the base rate), recall **66%** at the tuned threshold; delivery-time MAE **3.51 days** against **12.72 days** for Olist's own promised date. Models compared with 3-fold time-series CV: distance-rule baseline 0.605, HistGB 0.632, Random Forest 0.635, Logistic Regression **0.647** (selected).

9.2 Features under development

Confirmation re-run of TC-02/TC-24 on the current Kaggle files; LightGBM in the model comparison (optional dependency); items listed in section 14.

9.3 Screenshots of the working system

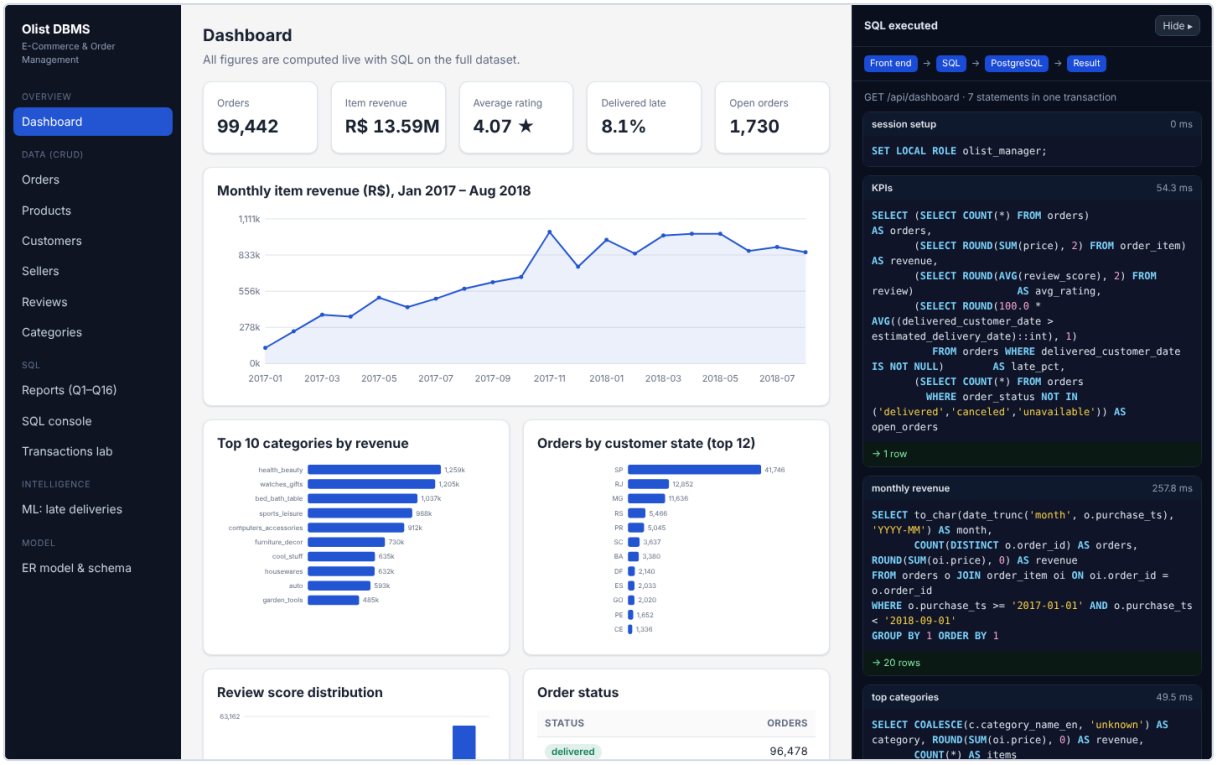


Figure 4: Dashboard with the live "SQL executed" panel (Front end → SQL → Database → Result).

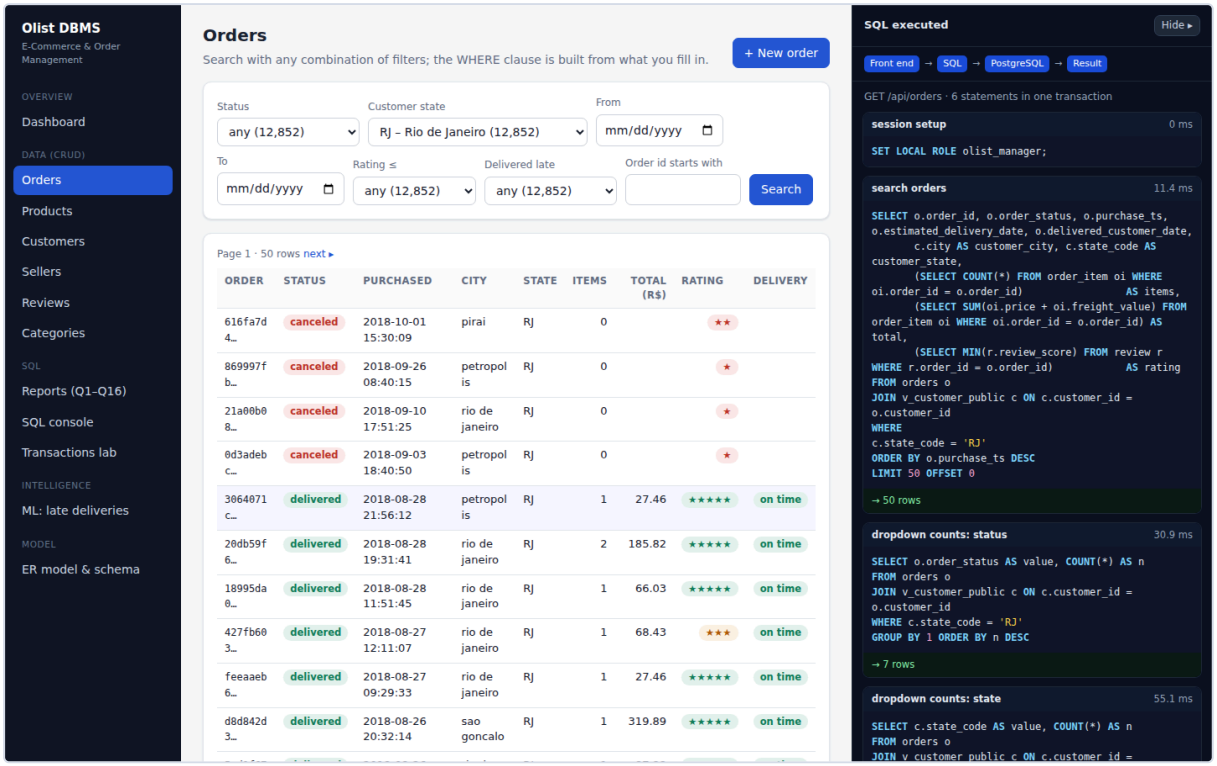


Figure 5: Faceted order search: each dropdown shows how many rows each option returns.

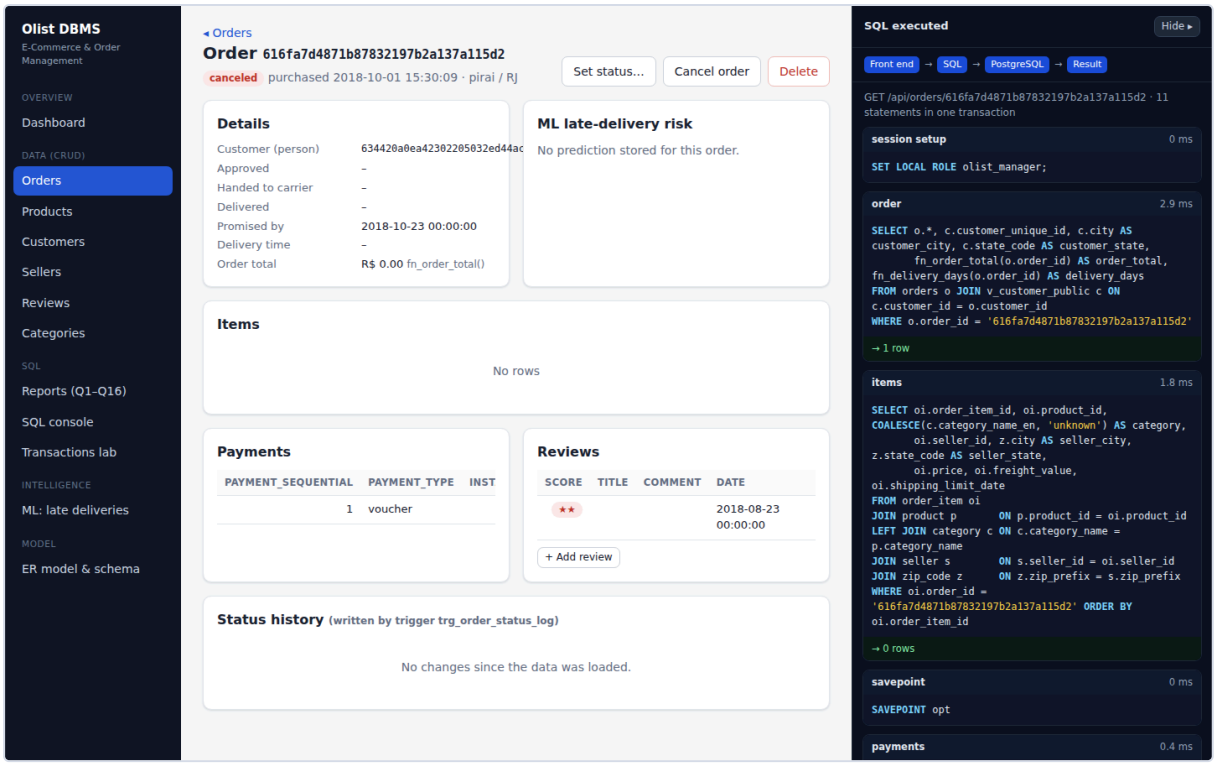


Figure 6: Order detail with ML risk and the audit trail written by the trigger.

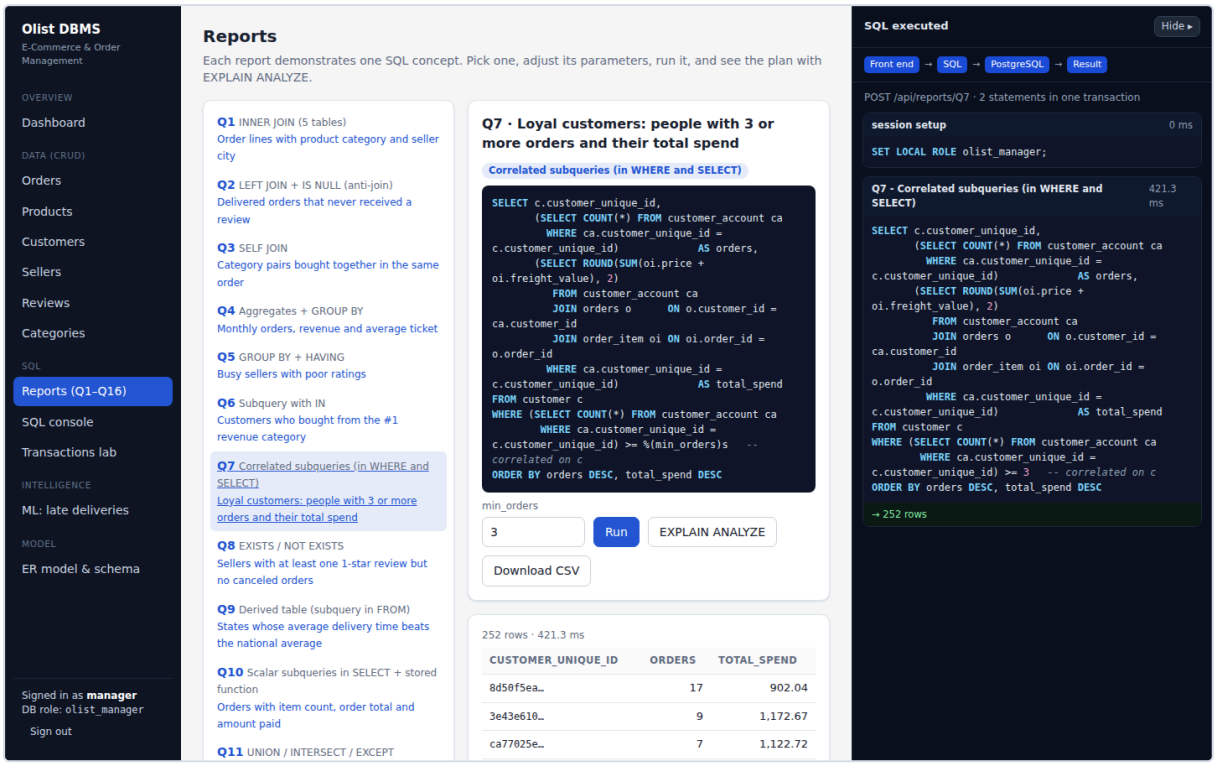


Figure 7: Reports Q1–Q16 with EXPLAIN ANALYZE and CSV export.

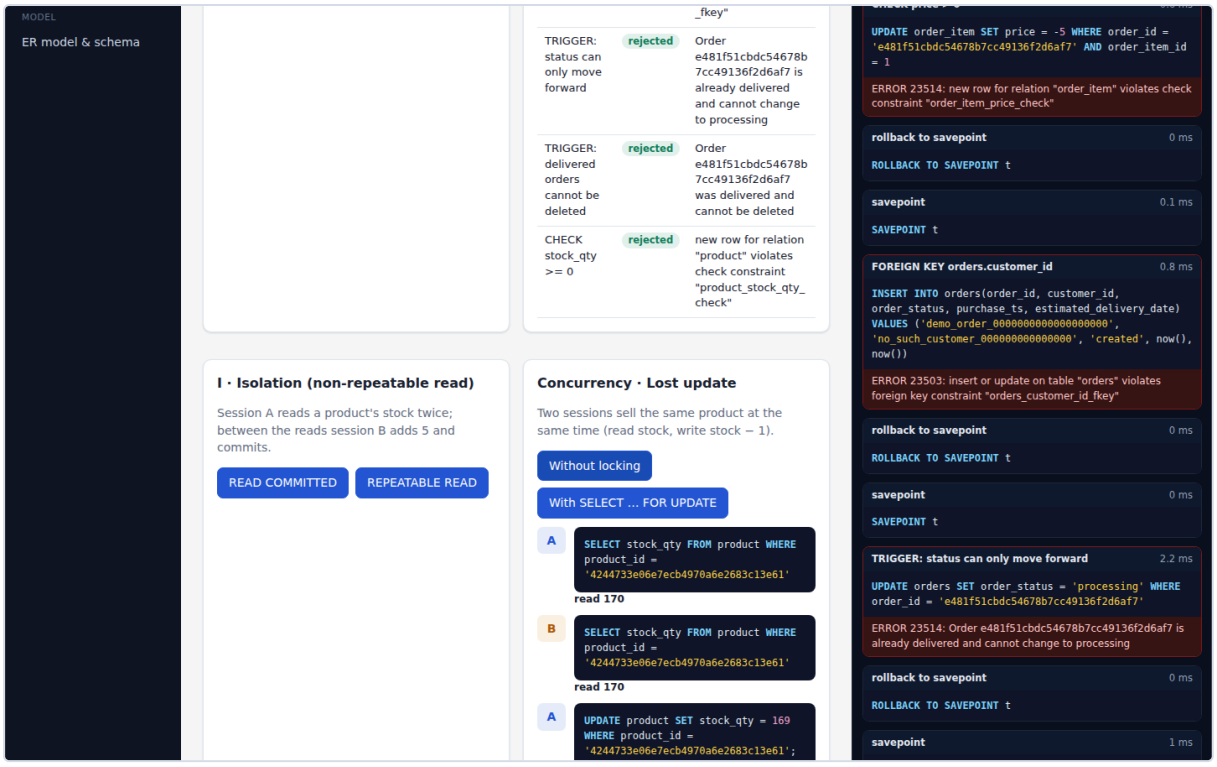


Figure 8: Transactions lab: atomicity, consistency, isolation levels and lost update.

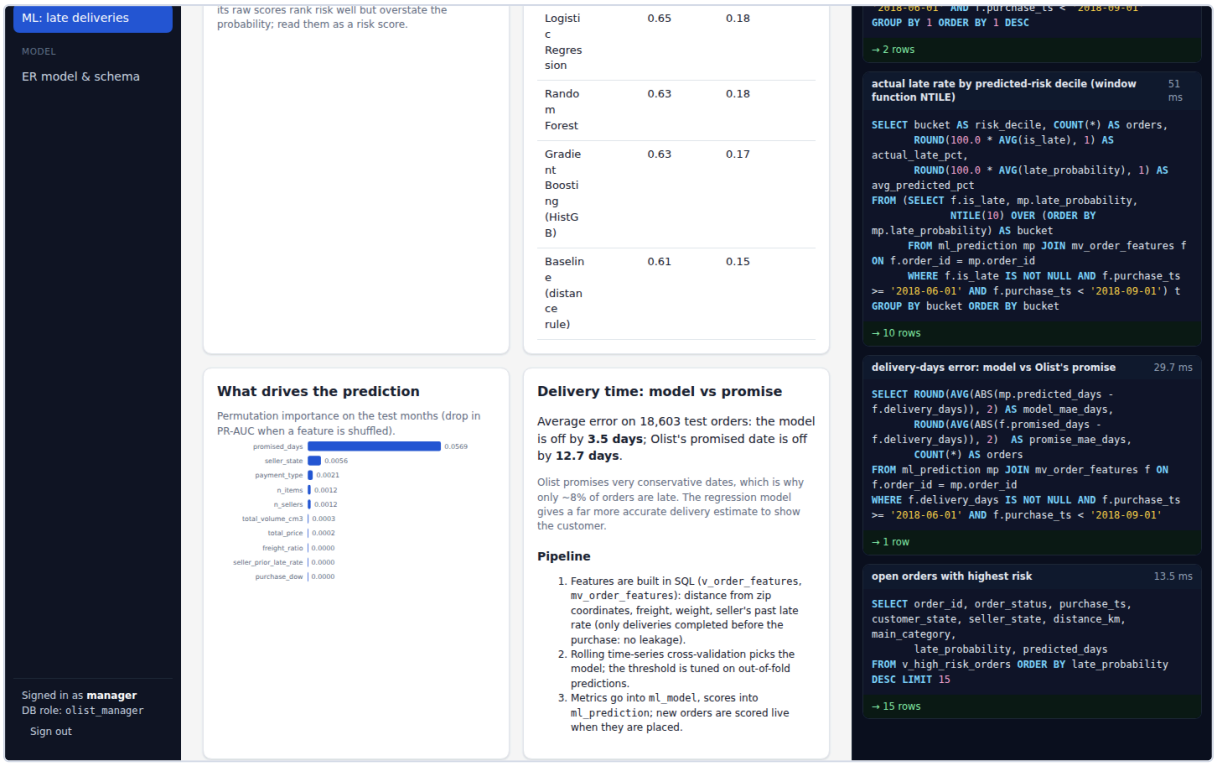


Figure 9: ML page: model metrics, confusion matrix and risk deciles computed in SQL.

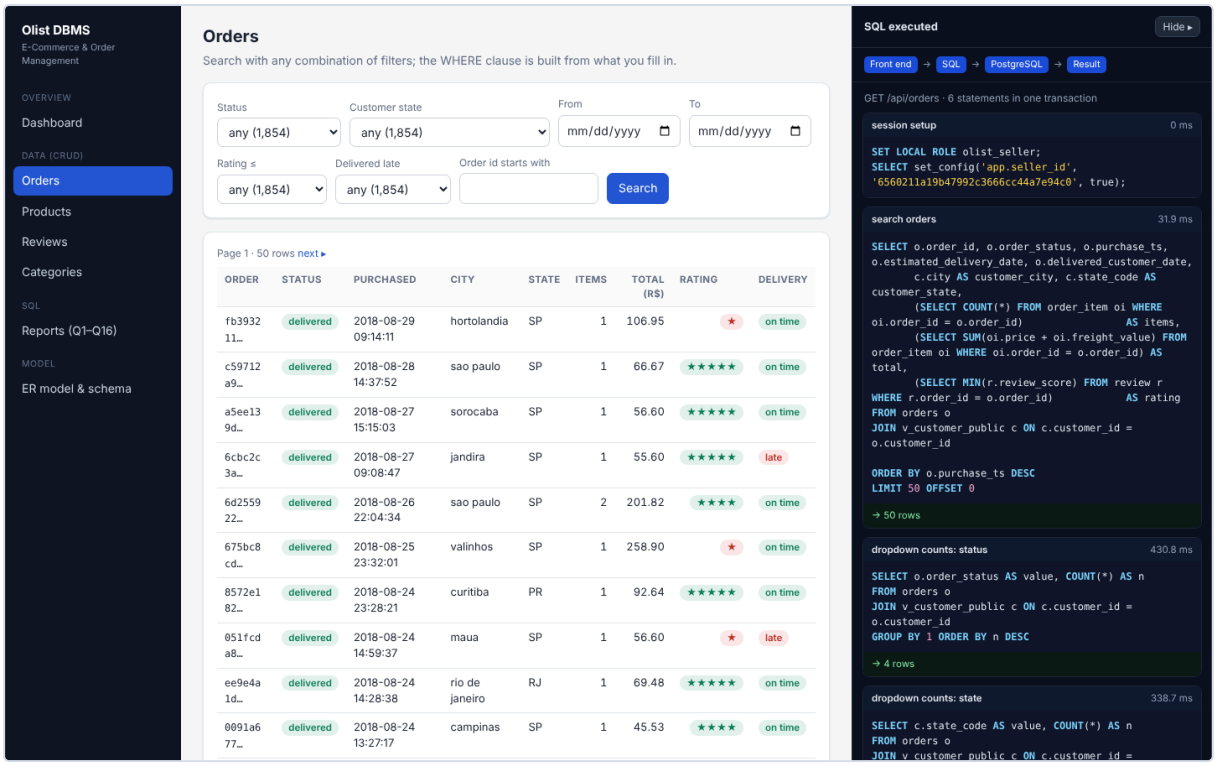


Figure 10: Logged in as a seller: row-level security limits every query to 1,854 of 99,441 orders.

9.4 Important implementation decisions

- **Rules in the DBMS, not Flask.** They also hold for `psql` and the ML script.
- **Per-request `SET LOCAL ROLE`** instead of one powerful app user. The database enforces least privilege and RLS.
- **No ORM.** The SQL must be visible; client-side binding keeps it safe from injection and exact.
- **Text-only staging schema.** All CSVs load without rejects; cleaning happens in SQL, and anomalies stay visible in `v_data_quality` (1,381 rows) instead of being silently dropped.
- **ML task chosen after profiling.** Recommendation was dropped because only 3.1% of customers buy twice. Late delivery is learnable and useful.
- **Leakage control.** Features use only information known at purchase. Seller history counts only deliveries completed before the purchase. Validation is time-ordered.

10. Testing performed

Test cases were designed against the SRS (`docs/TestPlan.md`) and executed on 15 Oct 2026 with two automated scripts: `db/15_test_suite.sql` (database rules, every test inside `BEGIN … ROLLBACK`) and `tests/api_tests.py` (end-to-end through the running app). The full table with inputs, expected and actual results is in `05_Testing/Test_Cases.pdf`, and logs and screenshots are in `05_Testing/Test_Evidence/`.

Area	Test cases	Passed	Note
Database build and data	TC-01 – TC-05	5	TC-02: review count reflects the dataset copy used
Business rules	TC-06 – TC-14	9	
Security and RBAC	TC-15 – TC-21	7	
Front end and reports	TC-22 – TC-28	7	TC-24: Q2 returns 0 rows on the older dataset copy
ML component	TC-29 – TC-31	3	
Total	**31**	**31**	2 with a dataset-version note, 0 failed

Sample of the test-case table

ID	Input	Expected	Actual	Status
TC-06	Change a delivered order to `shipped`	Error, unchanged	"Order ... is already delivered and cannot change to shipped"	Pass
TC-13	Order 2 units of a product with stock 1	CHECK fails; whole order rolled back	`product_stock_qty_check` violated; stock still 1; 0 orders added	Pass
TC-17	Seller opens Orders	Own orders only	1,854 of 99,441; 0 foreign orders	Pass
TC-20	Search with `' OR 1=1 --`	Literal, no extra rows	HTTP 400 on id filter; 0 rows on state filter	Pass
TC-27	Lost update without/with `FOR UPDATE`	One update lost / both kept	170 → 169 / 170 → 168	Pass
TC-28	Order search on full data	< 1 s	15–474 ms	Pass
TC-31	"Predict risk only"	Not saved	Order count 99,442 before and after	Pass

Defects: D-01 (home page 404) and D-02 (unclear error when PostgreSQL is down) were fixed on 29 Sep. Packaging found two minor issues: D-03, a test-plan wording fix for TC-19, and an apostrophe in an `\echo` in `db/13_index_demo.sql` that broke one heading (fixed in the submitted snapshot). No functional defects are open.

11. Project-management status

Process: Agile Scrum, four sprints (Sprint 0: 18–19 Sep requirements and ER; Sprint 1: 20–23 Sep database; Sprint 2: 24–27 Sep API, UI, ML; Sprint 3: 28 Sep–1 Oct hardening and release). Plan, WBS, Gantt chart, COCOMO estimate (upper bound ≈12.8 person-months vs ≈1.5 actual) and risk register: `06_Project_Management/Project_Plan.pdf`.

Current progress: all planned mid-semester features are done and tested.

Delayed / pending task	Reason	New date
Test execution with recorded results (planned 1 Oct)	Test plan written 28 Sep, but not executed before the sprint closed	Done 15 Oct
Re-run TC-02 / TC-24 on current Kaggle files	Test machine used an older public dataset copy	Before the demo

Top risks and mitigation

Risk	Mitigation	Status
R2: ML task not learnable	Profile first; baseline; honest metrics	Occurred: task changed to late delivery
R3: Data leakage inflates metrics	Time-based split; only past seller history	Controlled
R6: Environment differences (Windows paths, PostgreSQL not running)	`.bat` scripts check prerequisites; troubleshooting table	Occurred once; fixed
R7: Schedule slip before deadline	Must-have vs nice-to-have list	Test execution slipped two weeks; recovered

12. Individual contribution table

Member	Responsibilities	Commits
Tarush Banke	Planning and coordination; SRS; ER/EER model; schema; keys, FDs, BCNF; CSV load and cleaning; indexes; views; DDL/DML, index and normalization demos; design doc, README	25
Tanishk Varshney	Triggers; procedures and functions; roles, grants, column privileges, RLS; report queries Q1–Q16; ACID and concurrency demos; Flask API and DB access layer	24
Tanishq Mahajan	Front end; ML framing, features, training, evaluation and integration; test plan, testing and screenshots	18

Task allocation, weekly progress, full Git history and the signed declaration: `06_Project_Management/Contribution_Record.pdf`.

13. Challenges faced and solutions attempted

Challenge	Solution
Dirty geolocation data (1,000,163 points, outliers outside Brazil, 278 + 7 missing zip prefixes, untranslated categories)	Text-only staging; one cleaned point per zip prefix; coordinate CHECKs; missing zips and categories added; anomalies kept in `v_data_quality`
`review_id` is not unique	Composite key `(review_id, order_id)`, verified on data
Recommendation model not viable (3.1% repeat customers)	Switched to late-delivery prediction after profiling (risk R2)
Concept drift: monthly late rate swings from 1.4% to 21% (May 2018 truckers' strike)	Rolling time-series CV; test on untouched later months; honest caveat; the regression output is the more useful one
Seller status update must write the audit log, but sellers have no rights on `order_status_log`	Trigger functions `SECURITY DEFINER`
Showing the exact SQL safely	psycopg `ClientCursor` (client-side binding) + strict ID validation
Unclear failures on a laptop without PostgreSQL running (D-02)	`run_app.bat` checks `pg_isready` and explains the fix

14. Work planned for the remaining semester

Sprint	Planned work	Owner
4	Re-run TC-02/TC-24 on the current dataset; add LightGBM to the comparison; automate the test scripts in one command	Tanishq
4	Pagination/performance for large CRUD lists; more reports (seller cohort, freight analysis)	Tanishk
5	Backup/restore and a migration script for schema changes; partitioning experiment on `orders` by year	Tarush
5	High-risk order alerts in the UI (from `v_high_risk_orders`); model retraining from the UI	Tanishq, Tanishk
6	Usability review, final testing (regression of all TCs), final report and video	All

15. Disclosure of AI-tool use

Tool	Purpose	Nature of generated content	Verification / modification by students
Claude (Anthropic)	Assistance with parts of the code during development; writing the automated test scripts (`db/15_test_suite.sql`, `tests/api_tests.py`); creating the project report and submission documents	Code suggestions, test scripts, report text, diagrams and formatting	All members reviewed, tested and modified the content; the team ran the system and can explain every part

16. References

1. Olist, *Brazilian E-Commerce Public Dataset by Olist*, Kaggle, CC BY-NC-SA 4.0. kaggle.com/datasets/olistbr/brazilian-ecommerce
2. PostgreSQL Global Development Group, *PostgreSQL 16 Documentation* (row security policies, transaction isolation, pgcrypto, GIN indexes). postgresql.org/docs/16
3. IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications*.
4. R. Elmasri, S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson.
5. I. Sommerville, *Software Engineering*, 10th ed., Pearson.
6. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python", *JMLR* 12, 2011.
7. Pallets Projects, *Flask documentation*; psycopg 3 documentation (client-side binding cursors).
8. B. W. Boehm, *Software Engineering Economics* (COCOMO), Prentice Hall, 1981.